SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively. *(BL3)*

Assessment Tool Weightage: 35%

QUESTIONS: 3 **Duration : 1 Hour**
Total Marks:30

Annexure A

Constraint-Based Problem Solving

Code of Conduct:

I ***Chandra Prakash D / 192524060*** *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

Constraint-Based Problem Solving

1. A hospital needs to assign 3 doctors (D1, D2, D3) to 3 shifts (Morning, Afternoon, Night) such that:
   Constraints:
   i. D1 cannot work Night shift
   ii. D2 must work before D3 (Morning < Afternoon < Night)
   iii. Only one doctor per shift
   iv. D3 cannot work Morning
   v. All doctors must be assigned exactly one shift
   Apply Backtracking Search to find a valid assignment with all intermediate steps and constraint checks. State the final valid schedule

2. A robot operates in a grid environment (5×5). It must move from Start (S) to Goal (G). Obstacles block certain paths.
   Constraints:
   i. Movement allowed: Up, Down, Left, Right
   ii. Cost of each move = 1
   iii. Cannot pass through obstacles
   iv. Use Manhattan Distance heuristic
   Using above constraints and BFS Search Algorithm Show step-by-step node expansion and priority queue updates and determine the optimal path and cost.

3. An autonomous rescue robot is deployed in a disaster-affected building to locate and rescue survivors. The robot operates in a partially observable environment where some paths are blocked, and it must make decisions with limited information.

The building is represented as a grid of rooms. The robot starts at position S and must reach a survivor located at G.

Constraints:

- The robot can move up, down, left, right
- Some cells are blocked (obstacles) and cannot be traversed
- The robot has limited sensing capability (can only observe adjacent cells)
- Each movement has a cost of 1, but entering risky zones has an additional cost of 2
- The robot must minimize total cost while ensuring safe navigation
- The robot must update its knowledge dynamically (online search scenario)

Tasks:

b) Develop a solution approach using an appropriate search strategy (e.g., Uniform Cost Search / Online Search Agent). Clearly explain the steps involved.
c) Demonstrate how the robot applies AI concepts such as:

- Intelligent agents
- Rationality
- Environment type

d) Justify why your chosen approach is the most suitable for solving this problem under the given constraints.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

ANSWER SHEET

Artificial Intelligence (CSA17) – Assessment Tool 2: Constraint-Based Problem Solving

Name	Chandra Prakash D
Register No.	192524060
Course	Artificial Intelligence (CSA17) – SIMATS Engineering
Course Outcome	CO1 – Intelligent agents and uninformed search methods (BL3)

Question 1: Hospital Shift Assignment (Backtracking Search)

1.1 CSP formulation

Variables	D1, D2, D3 (the shift of each doctor)
Domain	{Morning (M), Afternoon (A), Night (N)} for every variable, with the order M < A < N
Search order	D1, then D2, then D3; values tried in the order M, A, N

ID	Constraint (from the question)	Formal form
C1	D1 cannot work Night shift	D1 ≠ N
C2	D2 must work before D3	shift(D2) < shift(D3)
C3	Only one doctor per shift	D1, D2, D3 all different (AllDiff)
C4	D3 cannot work Morning	D3 ≠ M
C5	Every doctor gets exactly one shift	Each variable takes one value (a complete assignment)

A constraint is checked as soon as all the variables it involves are assigned (C2 is therefore checked only when D3 is assigned). Any violated constraint makes the algorithm reject the value and **backtrack**.

1.2 Backtracking search – step-by-step trace

Step	Assignment tried	Constraint checks	Result
1	D1 = Morning	C1 ✓ (not Night); C3 ✓ (no other doctor assigned yet)	Consistent → go deeper
2	D1 = M, D2 = Morning	C3 ✗ (Morning already used by D1)	Reject, next value
3	D1 = M, D2 = Afternoon	C3 ✓ (M, A are different); C2 waits for D3	Consistent → go deeper
4	D1 = M, D2 = A, D3 = Morning	C3 ✗ (Morning used); C4 ✗ (D3 cannot be Morning)	Reject, next value
5	D1 = M, D2 = A, D3 = Afternoon	C3 ✗ (Afternoon used by D2)	Reject, next value
6	D1 = M, D2 = A, D3 = Night	C1 ✓ ; C2 ✓ (A < N); C3 ✓ ; C4 ✓ (N ≠ M); C5 ✓ (all assigned once)	SOLUTION ✓

Search tree (✗ = pruned, ✓ = solution):

```
root
└─ D1 = M  (C1 ok)
   ├─ D2 = M  ✗ C3
   └─ D2 = A  (C3 ok)
      ├─ D3 = M  ✗ C3, C4
      ├─ D3 = A  ✗ C3
      └─ D3 = N  ✓  all constraints satisfied  → STOP (first solution)
```

1.3 Remaining branches (if the search were continued)

Branch	Check	Result
D1 = M, D2 = N, D3 = M / A / N	D3 = M: C4 ✗; D3 = A: C2 ✗ (N < A false); D3 = N: C3 ✗	All rejected
D1 = A, D2 = M, D3 = N	C1 ✓ C2 ✓ (M < N) C3 ✓ C4 ✓	Second valid schedule
D1 = A, D2 = A or N	D2 = A: C3 ✗; D2 = N: no value of D3 works	Rejected
D1 = N	C1 ✗ (D1 cannot work Night)	Pruned immediately

There are exactly **two** valid schedules. Backtracking returns the first one it meets. Choosing the most constrained variable first (D1 ∈ {M, A}, D3 ∈ {A, N}) or applying forward checking would reach the answer with fewer rejected steps.

1.4 Final valid schedule

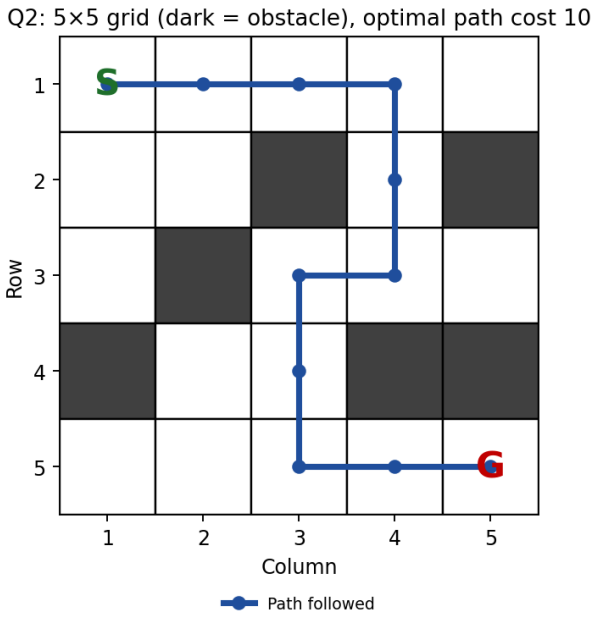
Morning	Afternoon	Night
D1	D2	D3

Result: D1 → Morning, D2 → Afternoon, D3 → Night. All five constraints hold (D1 not at Night, D2 before D3, one doctor per shift, D3 not in Morning, each doctor has exactly one shift).

Question 2: Robot on a 5 × 5 Grid (BFS and Manhattan-Distance Priority Queue)

2.1 Problem set-up and assumptions

The question does not print the grid, so the following 5 × 5 grid is assumed. Cells are written (row, column) with row 1 at the top. **S = (1,1), G = (5,5)**; obstacles at **(2,3), (2,5), (3,2), (4,1), (4,4), (4,5)**.



States	Free grid cells (19 free cells)
Actions	Up, Down, Left, Right (neighbour order used: U, D, L, R); moves into obstacles or outside the grid are not allowed
Step cost	1 per move
Goal test	Cell = (5,5)
Heuristic	Manhattan distance h(n) = |row − 5| + |col − 5| (admissible and consistent for 4-direction moves)

Note: Plain BFS uses a first-in-first-out queue and does not use a heuristic, while the question also asks for a priority queue and the Manhattan heuristic. Both are therefore shown: (A) BFS with its FIFO queue, and (B) the priority-queue version ordered by f(n) = g(n) + h(n) (A* search). Because every move costs 1, both give the same optimal answer.

2.2 (A) BFS – node expansion and queue updates

Visited cells are never added twice. The goal test is applied when a node is removed from the queue. g = number of moves from S.

Step	Node expanded (g)	New nodes added to queue	FIFO queue after the step (front → back)
1	(1,1) (g=0)	(2,1), (1,2)	(2,1) (1,2)
2	(2,1) (g=1)	(3,1), (2,2)	(1,2) (3,1) (2,2)
3	(1,2) (g=1)	(1,3)	(3,1) (2,2) (1,3)
4	(3,1) (g=2)	–	(2,2) (1,3)
5	(2,2) (g=2)	–	(1,3)
6	(1,3) (g=2)	(1,4)	(1,4)
7	(1,4) (g=3)	(2,4), (1,5)	(2,4) (1,5)
8	(2,4) (g=4)	(3,4)	(1,5) (3,4)
9	(1,5) (g=4)	–	(3,4)
10	(3,4) (g=5)	(3,3), (3,5)	(3,3) (3,5)

Step	Node expanded (g)	New nodes added to queue	FIFO queue after the step (front → back)
11	(3,3) (g=6)	(4,3)	(3,5) (4,3)
12	(3,5) (g=6)	–	(4,3)
13	(4,3) (g=7)	(5,3), (4,2)	(5,3) (4,2)
14	(5,3) (g=8)	(5,2), (5,4)	(4,2) (5,2) (5,4)
15	(4,2) (g=8)	–	(5,2) (5,4)
16	(5,2) (g=9)	(5,1)	(5,4) (5,1)
17	(5,4) (g=9)	(5,5)	(5,1) (5,5)
18	(5,1) (g=10)	–	(5,5)
19	(5,5) (g=10)	GOAL reached	[ ]

Optimal path found: (1,1) → (1,2) → (1,3) → (1,4) → (2,4) → (3,4) → (3,3) → (4,3) → (5,3) → (5,4) → (5,5), i.e. R, R, R, D, D, L, D, D, R, R. BFS removed 19 nodes (18 expanded + the goal).

2.3 (B) Priority queue with Manhattan distance (f = g + h)

The queue always gives the node with the smallest f first; ties are broken first-in-first-out. Entries are written node:f.

Step	Node expanded	g	h	f	Nodes inserted / updated	Priority queue after the step (front → back)
1	(1,1)	0	8	8	(2,1):8, (1,2):8	(2,1):8 (1,2):8
2	(2,1)	1	7	8	(3,1):8, (2,2):8	(1,2):8 (3,1):8 (2,2):8
3	(1,2)	1	7	8	(1,3):8	(3,1):8 (2,2):8 (1,3):8
4	(3,1)	2	6	8	–	(2,2):8 (1,3):8
5	(2,2)	2	6	8	–	(1,3):8
6	(1,3)	2	6	8	(1,4):8	(1,4):8
7	(1,4)	3	5	8	(2,4):8, (1,5):8	(2,4):8 (1,5):8
8	(2,4)	4	4	8	(3,4):8	(1,5):8 (3,4):8
9	(1,5)	4	4	8	–	(3,4):8
10	(3,4)	5	3	8	(3,3):10, (3,5):8	(3,5):8 (3,3):10
11	(3,5)	6	2	8	–	(3,3):10
12	(3,3)	6	4	10	(4,3):10	(4,3):10
13	(4,3)	7	3	10	(5,3):10, (4,2):12	(5,3):10 (4,2):12
14	(5,3)	8	2	10	(5,2):12, (5,4):10	(5,4):10 (4,2):12 (5,2):12
15	(5,4)	9	1	10	(5,5):10	(5,5):10 (4,2):12 (5,2):12
16	(5,5)	10	0	10	GOAL reached	empty

Observe that f stays at 8 (the Manhattan lower bound) along the cells next to the straight route. When (3,4) is expanded, the move to (3,3) has f = 6 + 4 = 10, because the walls force a detour. The queue then expands (3,3), (4,3), (5,3), (5,4) with f = 10 and finally removes G. The priority queue removed 16 nodes (15 expanded + the goal), three fewer than BFS.

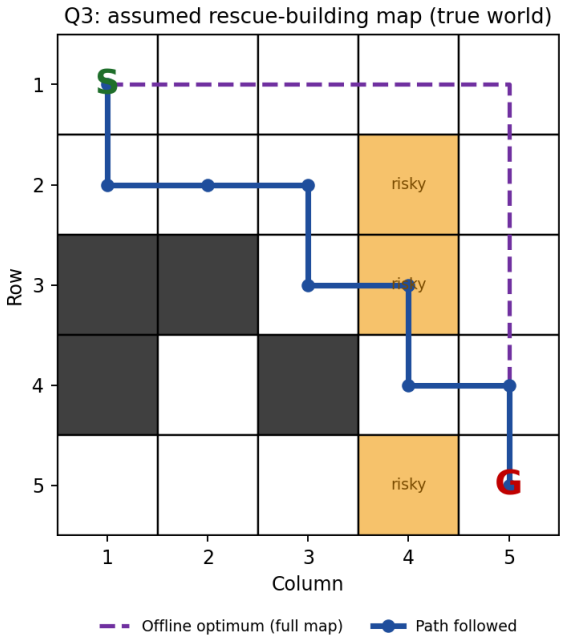
2.4 Result

Optimal path: S(1,1) → (1,2) → (1,3) → (1,4) → (2,4) → (3,4) → (3,3) → (4,3) → (5,3) → (5,4) → G(5,5). Optimal cost = 10 moves = 10 units.

The Manhattan distance from S to G is 8, so the true cost exceeds the heuristic by 2: the obstacles force one backward step (3,4) → (3,3). Both algorithms return the same cost, which confirms the result (BFS is optimal for uniform step cost, and A* is optimal because the heuristic is admissible).

Question 3: Autonomous Rescue Robot (Partially Observable, Online Search)

The question gives no map, so a sample 5 × 5 building is assumed to demonstrate the method. It has obstacles at (3,1), (3,2), (4,1), (4,3) and risky rooms at (2,4), (3,4), (5,4); S = (1,1) and G = (5,5). The robot does not know this map in advance – it only discovers a cell when it is adjacent to it. Entering a normal cell costs 1; entering a risky cell costs 1 + 2 = 3.



(a) Analysis of the constraints and their effect on decisions

Constraint	Effect on decision-making	How it is handled
Moves: up, down, left, right	Branching factor ≤ 4; no diagonal shortcuts, so Manhattan distance is a lower bound on cost.	Successor function generates at most 4 neighbours.
Blocked cells (obstacles)	Hard constraint: such cells can never be entered, so a plan can suddenly become invalid.	Blocked cells are removed from the known map and the plan is recomputed.
Sensing limited to adjacent cells	The environment is partially observable; the robot cannot plan on a full map and must act on beliefs.	Keep a knowledge map; unknown cells are assumed free (optimistic) until sensed.
Move cost 1, risky entry +2	Shortest path in steps is not always cheapest: a risky shortcut of 1 step costs as much as a 3-step safe path.	Edge cost = 1 (+2 if the entered cell is risky); use cost-based search.

Constraint	Effect on decision-making	How it is handled
Minimise cost with safe navigation	Two objectives: low cost and low danger. Safety is treated as more important than speed.	Obstacles are forbidden; risk is priced into cost, so risky cells are used only when the safe alternative is more expensive.
Dynamic knowledge update (online)	Decisions are interleaved with acting: plan → move → sense → update → re-plan.	Re-plan from the current position every time new information changes the map.

Priority of constraints: (1) hard constraints – obstacles and movement rules (never violated); (2) safety – avoid risky rooms where possible; (3) cost minimisation among the remaining feasible paths; (4) learning – update the map with every percept.

(b) Solution approach: Online Search Agent using Uniform Cost Search with re-planning

Steps involved:

- **Step 1 – Initialise:** knowledge map = {S}; current position = S; total cost = 0.
- **Step 2 – Sense:** observe the four adjacent cells and record each as free, blocked or risky in the map.
- **Step 3 – Plan:** run Uniform Cost Search on the known map from the current cell to G. Blocked cells are impassable, risky cells cost 3, and unknown cells are assumed free with cost 1.
- **Step 4 – Act:** execute only the first move of the plan and add its cost (1 or 3) to the total.
- **Step 5 – Update and repeat:** sense again; if the next planned cell is blocked or a cheaper route appears, re-plan. Repeat Steps 2–4 until the robot is at G.
- **Step 6 – Failure check:** if UCS finds no path on the known map, the survivor is unreachable with current knowledge, so report to the operator.

```
function ONLINE-UCS-AGENT(percept) returns an action
 persistent: map  // known cells: free / blocked / risky
             pos, plan, total_cost

 map <- UPDATE(map, percept)          // adjacent cells only
 if pos = G then return STOP
 if plan is empty or NEXT(plan) is blocked or a cheaper path exists then
    plan <- UNIFORM-COST-SEARCH(map, pos, G)
    //  step cost: free/unknown = 1, risky = 1+2 = 3, blocked = not allowed
    if plan = failure then return REPORT-UNREACHABLE
 action <- FIRST(plan);  plan <- REST(plan)
 total_cost <- total_cost + COST(action)
 return action
```

Demonstration on the assumed building

Step	Robot at	Newly sensed	UCS plan on known map	Plan cost	Move [cost]	Total so far
0	(1,1)	–	(1,1)→(2,1)→(3,1)→(4,1)→(5,1)→(5,2)→(5,3)→(5,4)→(5,5)	8	(2,1) [1]	1
1	(2,1)	(3,1) blocked	(2,1)→(2,2)→(3,2)→(4,2)→(5,2)→(5,3)→(5,4)→(5,5) (re-plan)	7	(2,2) [1]	2

Step	Robot at	Newly sensed	UCS plan on known map	Plan cost	Move [cost]	Total so far
2	(2,2)	(3,2) blocked	(2,2)→(2,3)→(3,3)→(4,3)→(5,3)→(5,4)→(5,5) (re-plan)	6	(2,3) [1]	3
3	(2,3)	(2,4) risky	(2,3)→(3,3)→(4,3)→(5,3)→(5,4)→(5,5)	5	(3,3) [1]	4
4	(3,3)	(4,3) blocked; (3,4) risky	(3,3)→(3,4)→(4,4)→(5,4)→(5,5) (re-plan)	6	(3,4) [3]	7
5	(3,4)	–	(3,4)→(4,4)→(5,4)→(5,5)	3	(4,4) [1]	8
6	(4,4)	(5,4) risky	(4,4)→(4,5)→(5,5) (re-plan)	2	(4,5) [1]	9
7	(4,5)	–	(4,5)→(5,5)	1	(5,5) [1]	10

Result: The robot reaches G with a total cost of 10 (8 moves + 2 extra for the one risky room, (3,4)). It re-planned 4 times (steps 1, 2, 4 and 6) as it discovered blocked or risky cells.

Had the whole map been known in advance, offline UCS would give the path (1,1) → (1,2) → (1,3) → (1,4) → (1,5) → (2,5) → (3,5) → (4,5) → (5,5) with cost 8, so the online robot paid 10/8 = 1.25 times the ideal cost. This extra cost is the price of limited sensing, because the robot could not know in advance that the left side of the building was blocked.

(c) Application of AI concepts

Intelligent agent

PEAS element	Rescue robot
Performance measure	Reach the survivor, minimise total path cost (1 per move + 2 per risky room), avoid unsafe cells, finish in minimum time.
Environment	Disaster-struck building modelled as a 5 × 5 grid of rooms with blocked and risky cells.
Actuators	Wheels or tracks: move Up / Down / Left / Right, stop, report status.
Sensors	Short-range sensors (proximity, camera or lidar) that observe the 4 adjacent cells; odometry for its own position.

The robot is a **model-based, goal-based and utility-based agent**: its internal map is the model, reaching G is the goal, and path cost (including risk) is the utility used to choose between routes. Its agent function maps the percept history (all cells sensed so far) to the next move; the agent program is ONLINE-UCS-AGENT above.

Rationality

A rational agent selects the action expected to maximise its performance measure, given the percept history and its built-in knowledge. In the demonstration the robot is rational at each step: at (3,3), after sensing (4,3) as blocked, the safe-looking routes were longer, so entering the risky room (3,4) at cost 3 gave the lowest remaining cost (6, versus 8 for going back around through row 1), and it avoided the risky rooms (2,4) and (5,4) when a cheaper cell existed. Rationality here does not mean perfection – it did not know in advance that the left side was blocked, but it never chose an action that was worse than the best one for what it knew.

Environment type

Property	Value	Reason
Observable	Partially observable	Only adjacent cells can be sensed.
Agents	Single agent	Only the robot acts (survivors and debris are treated as part of the environment).
Deterministic?	Deterministic (assumed)	A move to a free cell always succeeds; uncertainty comes from missing knowledge, not from the actions.
Episodic?	Sequential	Each move changes the position and affects later costs.
Static?	Static (assumed)	Obstacles do not move while the robot plans; if they collapse later, re-planning still handles it.
Discrete?	Discrete	Finite grid cells and four actions.
Known?	Unknown	The map is unknown at the start, so an online (exploration) search is required.

(d) Why this approach is the most suitable

Alternative	Why it is less suitable
Offline UCS / A* on the full map	Needs the complete map beforehand, which a partially observable disaster site does not provide.
BFS	Optimises the number of steps, not cost; ignores the +2 risk penalty and can send the robot through risky rooms.
DFS	Not optimal and may wander deeply into dead ends; wastes movement in a dangerous place.
Greedy best-first	Follows the heuristic only, ignoring risk cost; can get trapped behind obstacles and is not optimal.
Random walk / pure reflex	Cannot guarantee reaching G or minimising cost, and is unsafe.

Online UCS with re-planning is chosen because:

- **It works with limited information:** planning is done on the knowledge gathered so far and corrected after every sensing step.
- **It respects the cost model:** UCS expands the cheapest path first, so risky rooms (cost 3) are used only when no cheaper safe path exists.
- **It is optimal on the known map and complete:** with positive step costs, UCS finds the least-cost path whenever one exists on the known map; if the survivor is truly reachable, the robot eventually gets there.
- **It is safe:** blocked cells are hard constraints and the robot never enters a cell it knows is blocked.
- **It is simple and robust:** needs no heuristic or prior map, and re-planning absorbs new obstacles or changes in risk.

Limitation and improvement: re-planning from scratch every step repeats work. For larger buildings, incremental methods such as D* Lite, or A* with a Manhattan heuristic over the known map, give the same behaviour more efficiently.